What is the Big O?

Friday, October 23, 2026 2:33 AM

CORE

Programming problem. Everyone attempts it.

```
n = 5  # example value, you can change
for i in range(0, n):
    count = 0
    x = i * n
    print("i, x:", i, x)
    for j in range(0, x):
        count += 1
        print("count:", count)
```

i Iteration	i	count	x	j Iteration	count
1	0	0	0*5 = 0	range(0,0) → doesn't run	
2	1	0	1*5 = 5	1	1
				2	2
				3	3
				4	4
				5	5 = 1*5 = K*n
		reset ↓			
3	2	0	2*5 = 10	1	1
				2	2
				⋮	⋮
				10	10 = 2*5 = K*n
4	3	0	3*5 = 15	1	1
				2	2
				⋮	⋮
				15	15 = 3*5 = K*n
5	4	0	4*5 = 20	1	1
				2	2
				⋮	⋮
				20	20 = 4*5 = K*n

K = outer loop index (i)

range(0,5)

inner loop over range(0,0)

inner loop over range(0,10)

inner loop over range(0,15)

inner loop over range(0,20)

of j iterations: 5
n*1
n*K iterations

of j iterations: 10
n*2
n*K iterations

of j iterations: 15
n*3
K*n iterations

of j iterations: 20
n*4
K*n iterations

Output:
```
i, x: 0 0
i, x: 1 5
count: 1
count: 2
count: 3
count: 4
count: 5
i, x: 2 10
count: 1
count: 2
⋮
count: 10
i, x: 3 15
count: 1
count: 2
⋮
count: 15
i, x: 4 20
count: 1
count: 2
⋮
count: 20
```

outer i loop terminates: range(0,5)

outer loop stops when K = n

of outer iterations: n

Cost one 1 iteration:

$K = K \cdot n \times \alpha(1) = K \cdot n$ (also the # of prints)

Total cost T(n):

T(n) = cost of pass 0 + cost of pass 1 + ... + cost of pass (n-1)

$T(n) = \sum k \cdot n$ (k=0 to k=n-1)

$= n \cdot \sum k$

$= n \cdot n(n-1)/2$

$= n^2(n-1)/2$

$= \frac{n^3 - n^2}{2}$

When simplifying T(n) to O(n), drop everything BUT the biggest term

T(n) → O(??)

$\frac{n^3 - n^2}{2}$ → $O(n^3)$

The time complexity is $O(n^3)$!